SES598 HW3: Rocky Times Challenge - Search, Map, and Analyze

Gianna Bowden

16 March 2026

1 Background

In the README.md provided in the forked repository [1], the main goal of the assignment is to implement an autonomous drone system for geological feature detection, mapping, and analysis using an RGBD camera and PX4 SITL simulation. A controller is tuned for a PX4-powered drone to efficiently search, map, and analyze 3D objects in an unknown environment. The drone must map two cylinders and land on the top of the taller cylinder.

The drone utilizes computer vision to create a mask of the image and detect the cylinder, and publish the estimate of the cylinder size. The drone is also able to detect the aruco markers and their position from the drone camera.

2 Simulation Setup

Arguably, the setup of this simulation is rather intensive. In addition to the code files shared in the forked repository [1], there are some additional requirements. These additional requirements are explained below.

2.1 QGroundControl

Upon initial simulation testing, it was seen that as part of pre-arming activities from the drone, there is a check for ground control station connectivity. Without this connection, the drone will not arm and continue the take off process. To resolve this, the "QGroundControl" [2], [3] station was installed. The following commands were ran to install QGC, please note that libfuse2 could be installed first and a newer version of QGC could be selected. To launch QGC, `./QGroundControl.AppImage` needs to be ran in the terminal.

```
wget https://github.com/mavlink/qgroundcontrol/releases/download/v4.4.3/
QGroundControl.AppImage
chmod +x QGroundControl.AppImage
./QGroundControl.AppImage
sudo apt install libfuse2
```

2.2 PX4 Setup

The PX4 Software in the Loop (SITL) is the software that is used on the drone. Please note the top level course README.md [1] provides information on installing the PX4 dependencies.

```
git clone https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot
git checkout 9ac03f03eb
bash Tools/setup/ubuntu.sh
```

2.3 State Flow

The drone within the simulation follows a dedicated flow in order to fly autonomously and land. The following states are defined below:

- WAIT INTRINSICS: Waits for the camera intrinsics and the battery level, providing up with the initial battery level at take off.
- ARM TAKEOFF: Arms the drone and allows it to takeoff to the initial z direction of -5.0, it then files to the entry point of the circle at [-15, 0, -5]
- CIRCLE: After reaching the entry point, the drone keeps a constant altitude and starts to move in a circle at a speed of 0.02 rad/sec. This continues until a cylinder is detected in the image.
- SERVO: Once a cylinder is detected, the drone moves so it is 15m away from the cylinder.
- HOVER: Once the position is reached, the drone will hover by holding the position for 5 seconds and calculate the cylinder dimensions. The drone also checks to see if this measurement was calculated before. If not, the drone will continue the circle pattern until a new detection. If so, it will proceed to ARUCO HOVER.
- ARUCO HOVER: Once a repeat measurement is made, the drone will fly above both cylinder to around -20m. This position is held for 7 seconds.
- ARUCO SELECT: The landing distance to the tallest cylinder aruco marker is calculated.
- ARUCO MOVE: The drone flies to the target aruco marker. When the drone is 1.5m, the landing sequence is initiated to land straight down.
- ARUCO LAND: Once landed, the drone disarms.
- COMPLETE: The battery at mission end, battery used, and mission time is determined as this mission is now complete.

2.4 Image Processing

In general, the image processing for both the detection of the cylinders and the aruco markers. On the front of the drone there is a RGB-D camera that allows for the depth to be determined.

For the cylinder, the image is converted to OpenCV and processed into a black and white image. The noise in the image is minimized with the morphological process and the largest contour area gets a bounding box.

For the aruco markers, the camera on the bottom of the drone detects this. The position of these markers relative to the drone's camera are determined and this is then what the drone tracks in order to complete landing.

3 Initial Simulation

3.1 ARUCO Markers

The following Aruco Markers are located on the top of each cylinder. Marker 0 is located on the tall cylinder, and Marker 1 is located on the shorter cylinder.

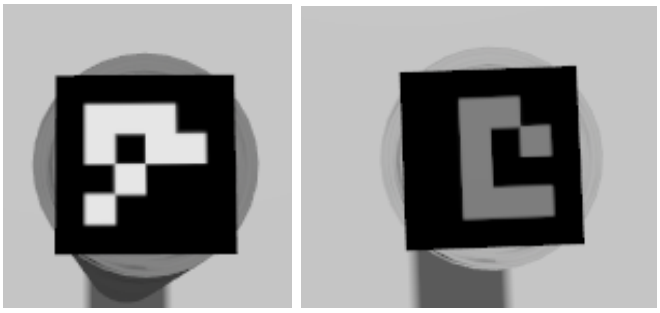

Figure 1: Cylinder Aruco Markers

3.2 Initial Cylinder Detection and Landing

The drone is initially able to detect the cylinders. However, the landing is inconsistent, there are simulations where the drone lands so closely to the edge that it falls off the cylinder.

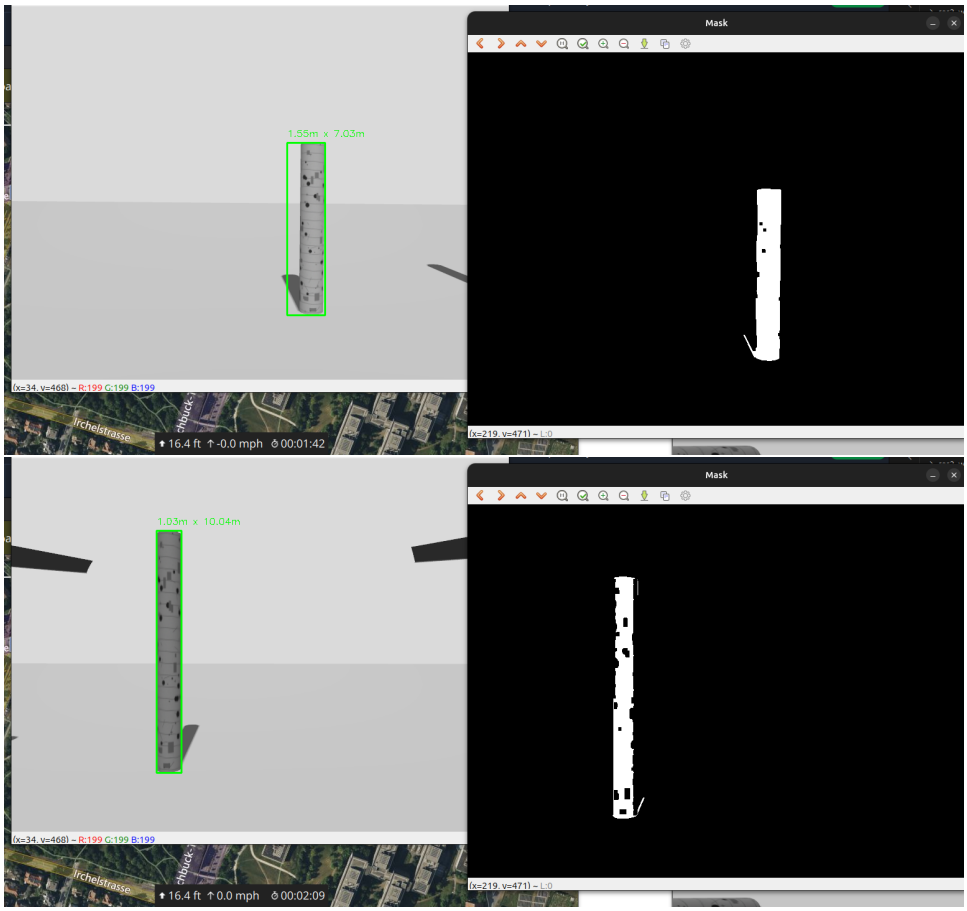

Figure 2: Cylinder Aruco Markers

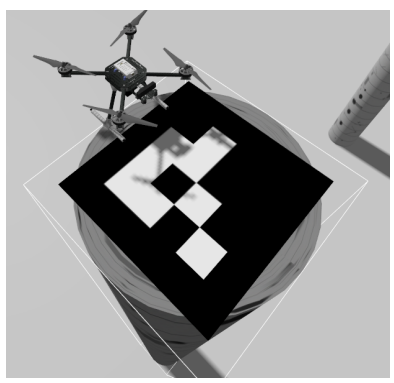

Figure 3: Cylinder Aruco Markers

3.3 Initial Simulation Performance

Below is the initial performance of the drone in the simulation. The drone detects both cylinders before moving to the aruco hover state, causing there to be an excess 30 seconds of simulation time.

Table 1: Initial Simulation Parameters - LANDED

Battery End	**Battery Used**	**Mission Time (s)**
0.9889	0.000 %	163.7
0.9889	0.000 %	164.89
0.9889	0.000 %	162.43

4 Tuned Simulation

The simulation parameters were tuned in order to complete the mission the fastest without compromising detection of the cylinders and ensure no undesired collisions. It was also important to ensure the drone landed close to the center of the To do this, the two main parameters modified were the distances to the aruco marker and the tolerance for cylinder detection.

There were minor offsets applied to the aruco marker 0 location detection. Specifically a value of 0.5 was added to the `dx` term and a value of -0.1 to the `dy` term. Additionally, the tolerance for the measured cylinders were increased to a value of 3. The drone struggled to measure the first cylinder height consistently. In order to increase the simulation time, it was key to get the drone to the aruco hover state after it senses the short cylinder again.

4.1 Tuned Simulation Parameters

Below are the parameters of the tuned simulation. The drone is now able to detect that the short cylinder is a repeat measurement as it completes the second circular rotation and is able to go to the hover state much sooner. This results in about a 30 second reduction in the time to complete the simulation.

Table 2: Initial Simulation Parameters - LANDED

Battery End	**Battery Used**	**Mission Time (s)**
0.9889	0.000 %	136.85
0.9889	0.000 %	134.13
0.9889	0.000 %	135.00

In each of the three trials above, the drone was able to land near the center of the cylinder. The landed drone in the tuned simulation is shown below.

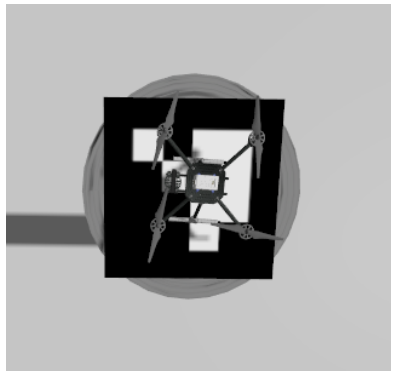

Figure 4: Cylinder Aruco Markers

5 Conclusion

In conclusion, the simulation mission time was greatly improved and completed successfully consistently. The main issue with the simulation initially was the mission time and that the drone often would not land near the center of the cylinder and would fall off. This was reminded by simply adding offsets to the tracked aruco marker positioning and increasing the measurement tolerance to ensure that only one of the cylinders was measured twice before hover.

Additionally, the battery used parameter did not seem to change during the simulation regardless of the mission time. This parameter was not able to be evaluated in this simulation in relation to improvements in mission time.

Please see the final video submission referenced below, under [4].

6 References

[1] Forked Repository: `https://github.com/gciaglia/ses598-space-robotics-and-ai-2026/tree/main/assignments/terrain_mapping_drone_control`

[2] Ground Control Documentation: `https://docs.qgroundcontrol.com/master/en/qgc-user-guide/getting_started/download_and_install.html`

[3] Ground Control Github: `https://github.com/mavlink/qgroundcontrol/releases`

[2]Video Submission: `https://github.com/gciaglia/ses598-space-robotics-and-ai-2026/blob/main/assignments/terrain_mapping_drone_control/FinalSubmission/TunedSimulation/FastDetection_SingleCylinder.mp4`